Data Communications and Networking

Assignment 2: Course Timetable Inquiry System

I. Introduction

Modern educational institutions rely on efficient digital systems to manage and deliver course information. A course timetable inquiry system enables students to conveniently access up-to-date scheduling information, while administrators can maintain and update records dynamically.

In this assignment, you will design and implement a client-server-based timetable inquiry system using Windows Socket Programming in C++. The system must support multiple concurrent users and provide reliable access to structured timetable data stored in a backend database.

II. Objectives

The objectives of this assignment are to develop a networked client–server application using socket programming, design and implement a structured database for course timetable management, and support concurrent client connections. In addition, the assignment aims to enable administrative control for system updates, implement basic system monitoring and logging functionalities, and apply key principles of reliable communication and robust error handling.

III. System Overview

The system follows a client-server architecture:

- Client: sends timetable queries and displays results,
- Server: processes requests, interacts with the database, and returns responses,
- Database: stores structured course timetable information.

The system should allow students to query course schedules anytime via a network connection.

IV. Functional Requirements

Your system must include the following modules:

(1) Database Module

- Store course timetable data, including: Course code, Course title, Section, Instructor, Time (day and duration), and Classroom.
- Data may be stored using: file-based storage (e.g., CSV, TXT), or embedded database (e.g., SQLite).

(2) Query Module (Client-Side)

- Allow users to: search by course code, search by instructor, view all courses for a given semester.
- Display results in a clear and readable format.

(3) User Management Module

- Support at least two roles: Student (default): query-only access; Administrator: full access.
- Administrator functions: add, update, and delete timetable records.
- Authentication (simple username/password) is required for administrator access.

(4) Information Update Module

- Allow administrators to: modify specific fields (e.g., time or classroom), add new course entries, and delete outdated entries.
- Changes must be reflected immediately for all connected clients.

(5) Networking and Concurrency Module

- Implement using Windows Sockets (Winsock) in C++,
- Support at least 5 concurrent client connections (minimum requirement),
- Use one of the following: Multithreading (e.g., std::thread), I/O multiplexing (e.g., select()).

(6) Communication Protocol

- Define a simple application-layer protocol, such as:

```
QUERY COMP3003
RESULT <course details>

LOGIN admin password
SUCCESS / FAILURE

UPDATE COMP3003 TIME Mon-10:00
OK
```

- Protocol must include: request format, response format, and error messages.

V. Non-Functional Requirements

Your system should be capable of handling invalid or malformed requests gracefully, preventing crashes caused by unexpected input, and ensuring a reasonable response time even under concurrent access. In addition, it should maintain a clean and modular code structure to support readability, maintainability, and future extensibility.

VI. Bonus Features

You may implement additional features for extra credit, such as developing a graphical user interface (GUI) for the client, enabling advanced search functionalities (e.g., searching by time slot), incorporating data caching to improve system performance, and implementing encryption or other mechanisms to support secure communication.

VII. Assessment Criteria

Component	Points
Overall Presentation	10
Overall Implementation	10
Bonus Features	10

VIII. Submission

(1) This is a group assignment, with up to four students per group.

(2) Students are required to submit the source code in a single compressed file via iSpace on or before **15 May 2026 (10:00 a.m.)**.

(3) Students are required to deliver a **10-minute presentation in English**, including a system demonstration, in Week 13 or 14.